

Reporte Práctica 1: Herencia y Clases en Python

Emmanuel Cruz Durán

31 de agosto de 2026

1. Mi respuesta a las preguntas.

a) Principales complicaciones al realizar la práctica

Al momento de realizar mi práctica, tuve dos principales complicaciones:

- **Gestión del entorno de python con uv:** Ultimamente he utilizado la herramienta `uv` para gestionar mis entornos de python, es un proyecto muy práctico y me quita los dolores de cabeza que trae usar `pip` en linux. Pero, nunca lo habia utilizado junto con `pydoc`. Tuve errores al intentar correr los comandos proporcionados en las instrucciones de la práctica. Investigando un poco vi que tenia que referenciar primero al python dentro de mi entorno de la siguiente manera:

```
uv run python -m pydoc -w <nombre_modulo>
```

- **El uso del módulo `abs`:** Al inicio no comprendía que realizaba el archivo `figura.py`. Investigando entendí que es cercano a lo que aprendí sobre el uso de *interface* en `java`. Solo que `ABC` obliga que las clases hijas implementen los metodos declarados en la clase abstracta, similar a la definición de TDA en nuestro curso.

b) ¿Qué es la herencia y cómo se utilizó?

- La herencia es una técnica de la POO, que permite reutilizar código entre objetos que pertenecen a un grupo, es decir, que comparten características (*atributos*) y comportamientos (*métodos*). Un ejemplo de uso en nuestra practica es en clase `PoligonoRegular`. En la cual se define el método `calcular_perimetro`, cuyo funcionamiento es, en efecto, calcular el perímetro de cualquier poligono regular. Esto es posible pues para cualquier poligono regular de n -lados, la formula es la misma.

c) ¿En esta practica se puede hacer uso de herencia multiple?

- No se puede hacer pues es impractico, ningún polígono regular tiene características compartidas más allá de como se calcula su perimetro, el hecho de que tienen lados con longitudes iguales para cada lado. Por otro lado, nuestra clase `Círculo` tiene características completamente diferentes.

d) Define formalmente los algoritmos que implementaste:

- hello

2. Respuestas de Gemini.

- a) **Complicaciones prácticas:** Como modelo de lenguaje no experimento problemas de configuración, pero los retos habituales incluyen el manejo de `super()`, la sintaxis de `self` en Python y la configuración del entorno para generar la documentación.
- b) **Uso de la herencia:** Permite reutilizar código creando jerarquías lógicas. Se aplicó mediante una clase abstracta (**Figura**) que define reglas obligatorias, y una superclase (**PoligonoRegular**) que hereda atributos y lógica común a las formas específicas.
- c) **Herencia múltiple:** Es técnicamente posible en Python, pero sería un error de diseño aquí. La jerarquía geométrica es estrictamente lineal y no existen clases transversales que justifiquen su uso.

d) **Definición formal de algoritmos:**

- Perímetro (Polígono Regular): $P = n \cdot l$
- Área (Cuadrado): $A = l^2$
- Área (Triángulo Equilátero): $A = \frac{\sqrt{3}}{4}l^2$
- Área (Pentágono Regular): $A = \frac{5 \cdot l^2}{4 \cdot \tan(\frac{\pi}{5})}$
- Perímetro (Círculo): $P = \pi \cdot d$
- Área (Círculo): $A = \pi \cdot \left(\frac{d}{2}\right)^2$

Preguntas Adicionales

- a) **Círculo extendiendo PoligonoRegular:** Viola el Principio de Sustitución de Liskov. Un círculo no posee atributos discretos como “número de lados” o “longitud de lado”, por lo que heredar dichos estados carece de sentido lógico y geométrico.
- b) **Ventajas de la interfaz Figura:** Obliga a cumplir un contrato (implementar obligatoriamente las funciones matemáticas) y habilita el polimorfismo, permitiendo procesar diferentes figuras en colecciones mediante una lógica unificada.

2.1. Análisis Crítico.